```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            },
            "venue_suggestions": [
                {{
                    "name": "",
                    "type": "",
                    "capacity": 0,
                    "estimated_cost": 0.0,
                    "search_terms": ""
                }}
            ],
            "remaining_budget": 0.0,
            "additional_suggestions": []
        }}

        Ensure all costs are in INR and total does not exceed the given budget.
        Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart, etc.
        """

        response = model.generate_content(prompt)
        result = extract_json_from_response(response.text)

        # Create INR calculation table
        result["calculation_table_inr"] = []
        categories = {}

        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "Misc")
            cat_allocation = category.get("allocation", 0)

            if cat_name not in categories:
                categories[cat_name] = {
                    "category": cat_name,
                    "items_count": 0,
                    "total_cost": 0,
                    "percentage_of_budget": 0
                }
```

- linking the websites like amazon,flipkar,ikea ect…. to get the products

```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            for item in category.get("items", []):
                categories[cat_name]["items_count"] += 1
                categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

            if result["total_budget"] > 0:
                categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

        for cat_data in categories.values():
            result["calculation_table_inr"].append(cat_data)

        # Define relevant shopping platforms for each category
        category_platforms = {
            "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
            "catering": ["swiggy", "zomato"],
            "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "decoration": ["amazon", "flipkart", "meesho", "myntra"],
            "entertainment": ["bookmyshow", "amazon", "flipkart"],
            "gifts": ["amazon", "flipkart", "myntra", "meesho"],
            "photography": ["google", "amazon", "flipkart"],
            "music": ["amazon", "flipkart", "bookmyshow"],
            "games": ["amazon", "flipkart"],
            "accessories": ["amazon", "flipkart", "myntra", "meesho"],
            "transportation": ["makemytrip", "google"],
            "return_gifts": ["amazon", "flipkart", "myntra", "meesho"]
        }

        # Default platforms if category is not in our predefined list
        default_platforms = ["amazon", "flipkart", "google"]

        # Add shopping links for each item
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
```

- using the if condition to categorize the situation based products

```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
                search_terms = item.get("search_terms", "")
                if search_terms:
                    item["shopping_links"] = {}

                    # Only add relevant platforms for this category
                    if "amazon" in relevant_platforms:
                        item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                    if "flipkart" in relevant_platforms:
                        item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bigbasket" in relevant_platforms:
                        item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                    if "swiggy" in relevant_platforms:
                        item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                    if "zomato" in relevant_platforms:
                        item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bookmyshow" in relevant_platforms:
                        item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "myntra" in relevant_platforms:
                        item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "meesho" in relevant_platforms:
                        item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "google" in relevant_platforms:
                        item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "booking" in relevant_platforms:
                        item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                    if "makemytrip" in relevant_platforms:
                        item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_
                    if "oyorooms" in relevant_platforms:
                        item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                    if "nobroker" in relevant_platforms:
                        item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search
```



```
    # Add search links for venue suggestions
    venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
    for venue in result.get("venue_suggestions", []):
        search_terms = venue.get("search_terms", "")
        if search_terms:
            venue["search_links"] = {}
            if "google" in venue_platforms:
                venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
            if "booking" in venue_platforms:
                venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
            if "makemytrip" in venue_platforms:
                venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(s
            if "oyorooms" in venue_platforms:
                venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
            if "nobroker" in venue_platforms:
                venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

- `/generate-jewelry`: Accepts text + image inputs to return style-matched jewelry options.
- `async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):`
- `    """Jewelry budget planner page"""`
- `    return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})`

- get home_recommendations by using the function

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
```

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
                "styling_tips": []
            }

            Make sure prices are in INR and stay within budget.
            Include Indian-friendly search terms for shopping.
            """
            response = model.generate_content([prompt, img])
        else:
            # Only text input
            prompt = base_prompt + """

            Format the output as JSON:
            {
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
                "styling_tips": []
            }

            Keep prices in INR and relevant to Indian brands.
            """
            response = model.generate_content(prompt)

        result = extract_json_from_response(response.text)

        # Add shopping links for each item (India-specific)
        for item in result.get("jewelry_recommendations", []):
            search_terms = item.get("search_terms", "")
```

- linking the websites like amazon,flipkar,ikea ect…. to get the products

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
        # Add shopping links for each item (India-specific)
        for item in result.get("jewelry_recommendations", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                    "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                    "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                }

        return result

    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```